

Revisao — Fase 2

Servico de Autenticacao (auth-service)

Projeto E-Commerce com Microservicos | Dev Junior | Davi x Claude

Sumario

1. O que foi construido na Fase 2
2. Bloco 1 — Estrutura base do servico
3. Bloco 2 — Banco de dados com Prisma
4. Bloco 3 — Cadastro e login com bcrypt
5. Bloco 4 — JWT: access e refresh token
6. Bloco 5 — RBAC: controle de papeis
7. Seed de admin idempotente
8. Bloco 6 — Rate limiting e seguranca
9. Bloco 7 — Testes automatizados
10. Fluxo de Pull Request adotado
11. Conceitos tecnicos consolidados
12. Historico completo de problemas e solucoes
13. Checklist de conclusao
14. Proximos passos — Fase 3

1. O que foi construído na Fase 2

A Fase 2 entregou o auth-service: o primeiro serviço real do projeto e o mais crítico de todos. Ele é a fundação de segurança da plataforma, pois todos os outros serviços dependerão dele para validar identidade e permissões. Um usuário não autenticado não acessa nada; um usuário sem o papel correto é bloqueado.

Entregável	Descrição
Cadastro de usuário	Senha hashada com bcrypt antes de persistir
Login	Retorna access token + refresh token
JWT	Access token (15min) + refresh token (7 dias)
Renovação de token	Endpoint /auth/refresh valida contra o banco
Middleware de autenticação	Protege rotas exigindo Bearer token
RBAC	Controle de acesso por papel: ADMIN, SELLER, BUYER
Seed de admin	Criação idempotente do admin inicial via env
Rate limiting	Proteção contra brute force no login
Testes	17 testes automatizados em 4 suites com Jest

Endpoints implementados

Endpoint	Acesso	Função
POST /auth/register	Público	Cadastro de novo usuário
POST /auth/login	Público (rate limited)	Login, retorna tokens
POST /auth/refresh	Público	Gera novo access token
GET /auth/me	Autenticado	Dados do usuário logado
GET /auth/admin/users	ADMIN	Lista todos os usuários
GET /health	Público	Health check do serviço

2. Bloco 1 — Estrutura base do servico

Inicializamos o auth-service como um projeto Node.js do zero, configuramos o TypeScript e montamos a arquitetura em camadas que separa responsabilidades.

Dependencias e o porque de cada uma (analogia da cozinha)

Pensando o servico como uma cozinha sendo montada, o npm install e a ida ao mercado comprar ingredientes. As dependencias de producao sao o que a cozinha usa todo dia; as de desenvolvimento sao ferramentas que so o cozinheiro usa, nao o cliente.

Pacote	Tipo	Funcao
express	Producao	O garcom: recebe requisicoes HTTP e direciona
dotenv	Producao	Le variaveis de ambiente do arquivo .env
bcrypt	Producao	Embaralha senhas de forma segura
jsonwebtoken	Producao	Gera e valida tokens JWT
express-rate-limit	Producao	Limita tentativas por IP
helmet	Producao	Adiciona headers de seguranca HTTP
typescript	Dev	Compilador que transforma .ts em .js
ts-node-dev	Dev	Roda TS direto e reinicia ao salvar
@types/*	Dev	Manuais de tipos das bibliotecas

Arquitetura em camadas

controllers (recebem HTTP), services (logica de negocio), middlewares (rodam entre a requisicao e o controller), routes (mapeiam URL ao controller), prisma (banco) e utils (funcoes auxiliares). Essa separacao permite testar a logica isolada do HTTP.

3. Bloco 2 — Banco de dados com Prisma

O Prisma é um ORM: traduz código TypeScript em SQL. Em vez de escrever queries manuais, trabalhamos com objetos e o Prisma cuida do SQL. Definimos o schema do usuário e rodamos a primeira migration para criar a tabela no PostgreSQL.

Conceitos centrais

Schema: define a forma das tabelas. Migration: arquivo SQL gerado a partir do schema que registra como o banco evoluiu. UUID como id (em vez de número sequencial) por segurança, para um atacante não adivinhar ids. Campo email com restrição única garantida pelo banco. Enum Role para limitar os valores válidos de papel.

-> Decisão técnica: fizemos downgrade do Prisma 7 para o 6. A v7 exige configuração em `prisma.config.ts`, adicionando complexidade desnecessária para o aprendizado. A v6 é estável e amplamente documentada.

4. Bloco 3 — Cadastro e login com bcrypt

Implementamos os endpoints de cadastro e login. O ponto central de segurança: a senha nunca é armazenada em texto puro. O bcrypt gera um hash antes de salvar, e no login compara o hash.

Por que bcrypt e o que é salt

O bcrypt é um algoritmo de hash criado para senhas. Diferente de SHA ou MD5 (rápidos), ele é intencionalmente lento, o que torna ataques de força bruta inviáveis. Ele também gera um salt automático: duas senhas iguais produzem hashes diferentes. O cost factor 10 define quantas rodadas de processamento — padrão da indústria, cerca de 100ms por hash.

Decisões de segurança

No login, tanto usuário inexistente quanto senha errada retornam a mesma mensagem `INVALID_CREDENTIALS`. Isso é intencional: mensagens diferentes permitiriam a um atacante descobrir quais emails estão cadastrados. O campo password nunca é retornado nas respostas da API, mesmo sendo um hash.

Separação controller e service

O controller cuida do HTTP: extrai dados da requisição, valida o formato e responde com o status correto. O service cuida da lógica: hash, consulta ao banco, regras. O controller não sabe como o hash funciona; o service não sabe que existe HTTP. Trocar o Express por outro framework não afetaria os services.

5. Bloco 4 — JWT: access e refresh token

Implementamos autenticacao stateless com JWT. O servidor nao guarda sessao em memoria: ele apenas valida a assinatura do token a cada requisicao.

Estrutura de um JWT

Um JWT tem tres partes separadas por ponto: header (algoritmo), payload (dados do usuario) e signature (assinatura). O payload nao e criptografado — qualquer um pode ler. A seguranca vem da assinatura: so quem tem o JWT_SECRET consegue gerar uma valida. Por isso nunca se coloca dado sensivel no payload, apenas id, email e role.

Por que dois tokens

Access token tem vida curta (15min) e vai em toda requisicao protegida; se vaziar, o estrago e limitado pelo tempo. Refresh token tem vida longa (7 dias), fica salvo no banco e serve apenas para gerar novos access tokens sem novo login. Usam secrets diferentes — um refresh token vazado nao pode ser usado como access token.

Middleware de autenticacao

O middleware roda antes do controller. Extrai o token do header Authorization (formato Bearer), valida a assinatura, e se valido injeta o userId na requisicao e chama next(). Se invalido, responde 401 e barra tudo. Reutilizavel em qualquer rota protegida.

6. Bloco 5 — RBAC: controle de papeis

RBAC (Role-Based Access Control) atribui permissões a papeis, não a usuários individuais. Todo BUYER pode X, todo SELLER pode Y, todo ADMIN pode Z.

Factory pattern no middleware

O `requireRole` é uma função que retorna outra função. Isso permite configurar quais papeis são aceitos em cada rota: `requireRole('ADMIN')` retorna um middleware que só deixa ADMIN passar. O papel vem do JWT, já extraído pelo middleware de autenticação.

Diferença entre 401 e 403

401 Unauthorized: você não provou quem é (token ausente ou inválido). 403 Forbidden: sei quem você é, mas você não tem permissão para isso. Essa distinção segue o padrão HTTP e foi testada nos três cenários: ADMIN acessa (200), sem token (401), BUYER bloqueado (403).

-> A ordem dos middlewares importa: `authMiddleware` primeiro (valida token e injeta o papel), depois `requireRole` (verifica o papel). Inverter quebraria, pois o papel ainda não existiria.

7. Seed de admin idempotente

Resolvemos o problema do bootstrap de admin: como todo cadastro nasce BUYER, sem uma solução intencional ninguém seria ADMIN para promover outros. O seed cria o admin inicial a partir de variáveis de ambiente.

A chave é a idempotência: o seed verifica se já existe algum ADMIN antes de criar. Se existe, não faz nada. Rodar o seed várias vezes é seguro. Em um deploy novo e limpo, ele cria o admin automaticamente.

8. Bloco 6 — Rate limiting e seguranca

Adicionamos protecao contra forza bruta. Sem limite, um atacante poderia tentar senhas infinitas no login. Criamos dois limitadores: global (100 req/15min) e login (5 tentativas/15min). Testado: a 6a tentativa de login e bloqueada com status 429.

Correcoes criticas vindas do code review

O review identificou tres problemas reais antes do merge. O mais grave: sem trust proxy configurado, atras do Nginx o servico veria apenas o IP do gateway, e 5 tentativas de uma pessoa bloqueariam o login para todos. Corrigido com `app.set('trust proxy', 1)`. Tambem movemos o `/health` para antes do limiter (monitoramento nao pode receber 429) e reordenamos o limiter global para rodar antes do parser de body.

9. Bloco 7 — Testes automatizados

Cobrimos os fluxos criticos com 17 testes em 4 suites usando Jest. Testes garantem que mudancas futuras nao quebrem o que ja funciona.

Suite	Testes	Cobertura
jwt.test.ts	3	Geracao, validacao e rejeicao de token invalido
password.test.ts	4	Hash, comparacao, rejeicao e salt
role.test.ts	3	RBAC: permite, bloqueia 403, bloqueia 401
authService.test.ts	7	register, login (feliz e falhas), refresh

Licao do code review sobre testes

O review apontou que testar `bcrypt` diretamente nao protege o codigo do service: se alguem removesse o hash do `registerUser`, o teste continuaria verde. Reescrevemos para testar `registerUser` e `loginUser` de verdade, mockando o Prisma. O teste mais valioso confirma que a senha persistida e hash, nunca texto puro. Tambem cobrimos o caminho feliz do login (gera tokens, salva refresh, nao vaza senha) e o `refreshAccessToken`.

Separacao de tsconfig

Tres contextos diferentes: `tsconfig.json` (editor, inclui tests com tipos do jest), `tsconfig.test.json` (Jest roda os testes), `tsconfig.build.json` (producao, sobrescreve types para node apenas, sem jest). Isso impede que globais de teste como `describe` vazem para o codigo de producao — validado: o build rejeita `describe` dentro de `src`.

10. Fluxo de Pull Request adotado

A partir do Bloco 4, adotamos o fluxo profissional de Pull Request: nenhum push direto na main. Cada bloco vira uma branch, é commitada, recebe push e abre um PR. O código é analisado (qualidade, segurança, performance, boas praticas) antes do merge.

O valor comprovado do code review

Os reviews encontraram problemas reais que teriam ido para producao: o bug do dotenv no seed (PR #3) que so apareceria no primeiro deploy; o trust proxy no rate limiting (PR #4) que derrubaria a autenticacao atras do Nginx; e o teste que nao protegia o codigo real (PR #6). Cada um foi corrigido antes do merge. Esse e exatamente o proposito do processo de PR.

PR	Conteudo	Resultado
#1	JWT (access + refresh token)	Aprovado com ajustes
#2	RBAC	Aprovado
#3	Seed de admin	Aprovado apos correcao (dotenv)
#4	Rate limiting	Aprovado apos correcao (trust proxy)
#5	Atualizacao do README	Aprovado (docs)
#6	Testes automatizados	Aprovado apos correcoes

11. Conceitos tecnicos consolidados

ORM (Prisma)

Traduz codigo para SQL automaticamente, com tipagem.

Migration

Arquivo SQL versionado que registra a evolucao do banco.

bcrypt

Hash lento de senha com salt automatico, contra brute force.

Salt

Valor aleatorio que faz senhas iguais gerarem hashes diferentes.

JWT

Token assinado com header, payload e signature.

Autenticacao stateless

Servidor valida assinatura sem guardar sessao.

Access token

Token curto (15min) para requisicoes protegidas.

Refresh token

Token longo (7d) salvo no banco, gera novos access tokens.

Middleware

Funcao que roda entre a requisicao e o controller.

RBAC

Controle de acesso baseado em papeis.

Factory pattern

Funcao que retorna outra funcao configurada.

401 vs 403

401: nao autenticado. 403: autenticado sem permissao.

Idempotencia

Operacao que pode rodar varias vezes com o mesmo efeito.

trust proxy

Faz o Express ler o IP real do cliente atras de um proxy.

Rate limiting

Limite de requisicoes por IP em janela de tempo.

Teste unitario

Testa uma funcao isolada do resto do sistema.

Mock

Substituto falso de uma dependencia (ex: Prisma) em teste.

12. Histórico completo de problemas e soluções

Todos os erros enfrentados durante a Fase 2, com causa, explicação e solução. Esta seção é a mais valiosa para consulta futura: quando esses erros aparecerem de novo, a solução estará aqui.

Erro: Prisma 7 – datasource url no longer supported (P1012)

Causa: Instalamos o Prisma sem fixar versão, veio a 7.x com breaking change.

Por que aconteceu: A v7 exige a URL de conexão em `prisma.config.ts`, não mais no `schema.prisma`.

Solução: Downgrade para Prisma 6 (`npm install prisma@6 @prisma/client@6`). A v6 é estável e bem documentada. Bonus: eliminou também 3 vulnerabilidades que a v7 trazia.

Erro: Authentication failed – credentials for postgres not valid (P1000)

Causa: O Prisma não conectava ao PostgreSQL mesmo com a senha correta.

Por que aconteceu: A `DATABASE_URL` usava 'localhost'. Rodando no WSL contra o container Docker, o endereço correto é 127.0.0.1. Além disso, redefinimos a senha do postgres dentro do container para garantir.

Solução: Trocar localhost por 127.0.0.1 na `DATABASE_URL` e rodar `ALTER USER postgres WITH PASSWORD`.

Erro: Código colado no terminal por engano

Causa: Uma estrutura de código TypeScript foi colada no terminal em vez do editor.

Por que aconteceu: Gerou uma série de erros de bash. O arquivo alvo continuou vazio.

Solução: Sempre criar arquivos via heredoc Python (`python3 - << 'PYEOF'`) ou direto no editor VS Code.

Erro: schema.prisma vazio / criado no diretório errado

Causa: O schema perdeu o conteúdo e depois foi recriado na raiz do projeto em vez de no `auth-service`.

Por que aconteceu: Os comandos Python rodaram no diretório errado. O Prisma Client ficou sem os tipos do modelo User.

Solução: Confirmar o diretório com `pwd` antes de criar arquivos. Recriar o schema dentro de `services/auth-service` e rodar `prisma generate`.

Erro: Cannot find module express / helmet

Causa: O TypeScript não encontrava os tipos do `express` e do `helmet`.

Por que aconteceu: Pacotes `@types` removidos durante o downgrade do Prisma; `helmet` 8 mudou o formato de exportação.

Solução: Reinstalar as dependências e os `@types`. Adicionar `allowSyntheticDefaultImports` no `tsconfig` para o `helmet` 8.

Erro: jwt.sign – No overload matches this call (TS2769)

Causa: O TypeScript não resolvia a tipagem do `expiresIn` no `jwt.sign`.

Por que aconteceu: A tipagem do `@types/jsonwebtoken` esperava um tipo específico para `expiresIn`.

Solução: Usar `SignOptions` explicitamente e fazer cast do `expiresIn` para `SignOptions['expiresIn']`.

Erro: `auth.routes` — has no exported member 'refresh'

Causa: A rota importava a função `refresh` do controller, que não existia.

Por que aconteceu: A função `refresh` não foi escrita quando o controller foi digitado à mão.

Solução: Adicionar a função `refresh` ao controller e garantir o import de `refreshAccessToken` do service.

Erro: `role.middleware` não criado (response em branco)

Causa: O BUYER não recebia resposta ao ser bloqueado; o arquivo do middleware não existia.

Por que aconteceu: O comando Python que cria o middleware rodou no diretório errado (raiz, não `auth-service`).

Solução: Rodar `cd` para o `auth-service` antes de criar o arquivo, e reiniciar o servidor para carregar o novo middleware.

Erro: `seed.ts` — Cannot find name '\$disconnect' / Invalid character

Causa: O seed não compilava por causa de uma barra invertida literal.

Por que aconteceu: O escape do `$` no heredoc Python deixou `prisma.$disconnect` no arquivo.

Solução: Corrigir para `prisma.$disconnect()` sem a barra invertida.

Erro: `npm run seed` — variáveis obrigatórias mesmo com `.env` preenchido (review PR #3)

Causa: O seed lia `process.env` mas as variáveis vinham undefined em ambiente limpo.

Por que aconteceu: O seed não chamava `dotenv.config()`. Funcionou no teste por coincidência (variáveis já no ambiente da sessão).

Solução: Adicionar `import dotenv` e `dotenv.config()` no topo do seed. Confirmado pelo 'injected env (9)'.

Erro: `.env.example` incompleto (review PR #3)

Causa: O `.env.example` só listava as variáveis `ADMIN_*`.

Por que aconteceu: O seed instancia o `PrismaClient`, então precisa de `DATABASE_URL`; o serviço usa `JWT_SECRET`.

Solução: Completar o `.env.example` com `DATABASE_URL`, `JWT_SECRET`, `JWT_REFRESH_SECRET` e os `ADMIN_*`.

Erro: Rate limit por IP atras do Nginx (review PR #4)

Causa: Sem `trust proxy`, o serviço veria apenas o IP do gateway.

Por que aconteceu: 5 tentativas de uma pessoa bloqueariam o login para todos os usuários por 15 minutos.

Solução: `app.set('trust proxy', 1)` para ler o IP real via `X-Forwarded-For`.

Erro: `/health` sujeito ao rate limit (review PR #4)

Causa: O `globalLimiter` rodava antes da rota `/health`.

Por que aconteceu: Health checks frequentes poderiam receber 429 e marcar o serviço como indisponível.

Solução: Mover a rota `/health` para antes do `globalLimiter` no `server.ts`.

Erro: Cannot find name 'describe' / 'it' / 'expect' (TS2593/2304)

Causa: O TypeScript não reconhecia as globais do Jest nos arquivos de teste.

Por que aconteceu: O tsconfig não incluía 'jest' no campo types, e a pasta tests estava fora do escopo.

Solução: Adicionar 'jest' aos types do tsconfig que o editor usa, mantendo o build de produção com types apenas 'node'.

Erro: File is not under rootDir 'src' (tsconfig)

Causa: Ao incluir a pasta tests, o TypeScript reclamou do rootDir restrito a src.

Por que aconteceu: O rootDir fixado em ./src entra em conflito com arquivos de teste fora dele.

Solução: Separar em três tsconfig: principal (editor), test.json (Jest, rootDir na raiz) e build.json (produção, so src).

Erro: jest types vazando para o build (review PR #6)

Causa: O build de produção aceitava globais de teste como describe dentro de src.

Por que aconteceu: O tsconfig.build.json herdava types com jest do principal sem sobrescrever.

Solução: Sobrescrever types: ['node'] no tsconfig.build.json. Validado: o build passa a rejeitar describe em src.

Erro: Teste não protegia o código real (review PR #6)

Causa: password.test.ts testava bcrypt diretamente, não o registerUser.

Por que aconteceu: Se alguém removesse o hash do service, o teste continuaria verde.

Solução: Criar authService.test.ts mockando o Prisma e testando registerUser e loginUser de verdade.

Erro: Cannot access 'mockFindUnique' before initialization

Causa: O mock do Prisma falhava por hoisting do Jest.

Por que aconteceu: O jest.mock é icado para o topo, antes das declarações const dos mocks.

Solução: Declarar as funções mock dentro do factory do jest.mock e expor via __mockFns.

Erro: RangeError: Invalid count value: -7 (reporter do Jest)

Causa: O Jest quebrava ao renderizar a barra de progresso.

Por que aconteceu: Bug do reporter: calcula a largura da barra com base no terminal; largura estreita da valor negativo.

Solução: Rodar com a flag --ci, que usa reporter simples sem barra animada. Fixado no script test do package.json.

Erro: forceExit mascarando vazamento (review PR #6)

Causa: O jest.config usava forceExit para silenciar o aviso de worker.

Por que aconteceu: O forceExit esconde a causa raiz de handles abertos em vez de corrigir.

Solução: Remover forceExit. Com o Prisma mockado não há conexão real aberta, o aviso some de raiz.

Erro: SSH connection timed out (porta 22 e 443)

Causa: O git push falhava por timeout na conexão SSH com o GitHub.

Por que aconteceu: A rede bloqueava a porta 22 (e também a 443 alternativa do SSH).

Solução: Trocar o remote de SSH para HTTPS: git remote set-url origin https://github.com/... Voltar a SSH em rede sem bloqueio.

13. Checklist de conclusao

[OK] Funcionando

Todos os endpoints respondem corretamente, testados manualmente e via curl

[OK] Testado

17 testes automatizados em 4 suites, todos passando

[OK] Commitado

6 PRs mergeados no padrao Conventional Commits

[OK] Documentado

README atualizado e este PDF de revisao

[OK] Compreendido

Conceitos validados ao longo dos blocos com explicacoes

[OK] Revisado

Cada bloco passou por code review antes do merge

[OK] Seguro

Senhas com bcrypt, JWT, RBAC, rate limiting, .env protegido

14. Proximos passos — Fase 3

A Fase 3 implementa o catalogo de produtos (product-service) e o controle de estoque (inventory-service). Sera o primeiro uso de MongoDB no projeto, justificado pela flexibilidade de schema para produtos com atributos variados por categoria.

O que vem pela frente

- CRUD completo de produtos e categorias com MongoDB via Mongoose
- Upload de imagens (Cloudinary ou S3)
- Busca, filtros e paginacao
- inventory-service para controle de estoque com PostgreSQL
- Cache de listagens de produtos com Redis
- Integracao entre product-service e inventory-service

Pendencias herdadas da Fase 2 (para retomar quando fizer sentido)

- Testes de integracao end-to-end com Supertest, incluindo o 429 do rate limit
- Rate limit com store no Redis (compartilhado entre instancias) ao escalar
- Paginacao no endpoint /admin/users
- Extender a interface Request do Express para evitar (req as any)
- Login social via OAuth2 com Google

Fase 2 concluida | auth-service completo e testado | Davi x Claude